

Gaussian Splatting SLAM based on Visibility Loop Closure and Depth Optimization

Yanfeng Xiao, Xinzhao Wu, Liwei Zhang*
School of Mechanical Engineering and Automation
Fuzhou University
 Fuzhou, China

*Corresponding author: lw.zhang@fzu.edu.cn

Abstract—Simultaneous Localization and Mapping (SLAM) is increasingly combined with 3D rendering techniques. We present a SLAM system that solely uses 3D Gaussians for scene representation. To enhance tracking, we propose a loop closure detection method based on visibility overlap of Gaussian point clouds, followed by pose-based filtering to reduce false positives. Loop closure detection triggers pose graph optimization to refine trajectories. For rendering, we apply multi-frame fusion to improve keyframe depth, and adopt monocular depth prediction when depth input is unavailable. Compared to MonoGS, our approach achieves approximately 26% improvement in tracking accuracy on TUM RGB-D and Replica datasets, and significantly enhances rendering quality on TUM RGB-D and Realsense datasets.

Keywords- SLAM, Depth Optimization, Loop Closure Detection, 3D Gaussian Splatting

I. INTRODUCTION

Simultaneous Localization and Mapping (SLAM) enables real-time localization in unknown environments. While traditional SLAM methods use geometric representations like point clouds, they struggle with high-fidelity reconstruction in complex scenes. Recently, 3D Gaussian Splatting (3DGS) [1] has shown near-photorealistic rendering using anisotropic Gaussians. Compared to implicit methods like NeRF[2], 3DGS offers faster rendering, better geometry, and higher efficiency—making it well-suited for SLAM applications.

Although 3DGS holds great promise for SLAM applications, its integration is still in early stages. Most existing methods focus on static reconstruction and overlook loop closure, a key component for long-term consistency. Without it, drift errors accumulate, degrading mapping accuracy, especially in large-scale scenarios.

Building upon the existing keyframe selection strategy in Gaussian Splatting SLAM, this work proposes a loop closure detection method based on visibility overlap. The detection process consists of two stages: a fast candidate selection using visibility mask similarity between Gaussian point clouds, followed by geometric distance verification to improve detection robustness. Once a loop is confirmed, we employ pose graph optimization via the g2o framework [16] to refine the camera trajectory globally. To enhance depth estimation, we introduce a multi-frame fusion approach that suppresses invalid depth values while improving the overall depth accuracy of keyframes. For monocular settings, we also improve the depth prediction strategy to enhance robustness under large viewpoint variations. Extensive experiments on indoor RGB-D datasets

demonstrate that our method significantly boosts both tracking accuracy and rendering quality for Gaussian Splatting-based SLAM systems. Our main contributions are summarized as follows:

- We introduce a visibility-overlap-based loop closure method, using Gaussian mask similarity for candidate selection and g2o-based pose graph optimization for drift correction and improved SLAM accuracy.
- We propose a multi-frame depth fusion strategy that suppresses invalid values and enhances keyframe depth accuracy, improving rendering quality.
- For monocular input, we enhance depth prediction using the MiDaS [27] model to improve accuracy under large viewpoint changes, boosting SLAM robustness.

II. RELATED WORK

A. Traditional SLAM mapping methods

Traditional SLAM methods like ORB-SLAM [3,4] use sparse or semi-dense features for localization but lack fine details for high-fidelity rendering. KinectFusion [6] introduced dense SLAM with GPU-accelerated TSDF fusion but suffers from memory scaling due to global voxel grids. ElasticFusion[7] supports dynamic scenes with deformation graphs and loop closure but is computationally heavy. BundleFusion [10] combines hierarchical pose graph optimization and depth fusion for improved global consistency but has high optimization costs, limiting real-time use on mobile devices.

B. Neural Radiation Field and 3D Gaussian Splattings

Neural Radiance Fields (NeRF) achieve near-photorealistic 3D reconstruction via implicit scene encoding using MLPs. iMAP [11] first integrated NeRF into SLAM, jointly optimizing poses and implicit scenes through online training, but its high computation time limits real-time use. 3DGS [1] overcomes this by explicitly representing scenes with anisotropic Gaussians and differentiable rasterization, enabling real-time rendering and efficient GPU optimization. Compared to NeRF-based methods that require heavy MLP inference and slow training for every new scene, 3DGS naturally supports incremental updates and faster convergence, making it more suitable for real-time SLAM applications. Recent works have further extended 3DGS to practical scenarios such as AR/VR navigation and robotic exploration, where low-latency dense mapping is essential. These capabilities position 3DGS as a promising solution for SLAM systems demanding both visual quality and

computational efficiency. MonoGS [14] applies 3DGS to monocular SLAM with keyframe selection and Gaussian optimization but lacks loop closure, causing drift in long sequences.

C. Loop closure detection

Traditional visual SLAM uses Bag-of-Words models like DBoW2 [15] for fast image retrieval but suffers from high false positives in low-texture or changing viewpoints. FAB-MAP [17] improved robustness with Chow-Liu trees modeling visual word co-occurrence. With deep learning, loop closure detection now relies on learning-based global descriptors like NetVLAD [18], which combines CNN with VLAD pooling for robust features but incurs higher computational and storage costs.

To further improve robustness, Sünderhauf et al. [19] integrated semantic object info into maps, enhancing loop closure stability. Porzi et al. [20] used semantic consistency graphs to reduce false matches from repetitive structures. Rosinol et al. [21] introduced 3D semantic co-views for dynamic scene loop detection. NeRF-based SLAM [8,9,11,12,13] methods like NICE-SLAM [8] generate loop candidates via submap matching but require costly online training of multiple local models. LiDAR SLAM [22,23] verifies geometry with ICP/NDT but depends on point density and is sensitive to noise, limiting use in monocular or RGB-D data.

D. Depth prediction

Early depth estimation relied on supervised learning, such as Eigen et al.'s [24] multi-scale CNN trained on labeled data. Due to data cost, unsupervised methods emerged: Zhou et al. [25] used image reconstruction loss for monocular videos, and Godard et al. [26] introduced left-right consistency. Ranftl et al.'s MiDaS [27] model trains on diverse datasets for strong cross-domain monocular depth prediction, emphasizing relative depth consistency. Multi-view stereo networks like MVSNet [28] use cost volumes for dense depth. Depth prediction has been integrated into SLAM: DynaSLAM [29] combines

semantic segmentation and depth to handle dynamics, while NICE-SLAM [8] fuses depth with implicit neural scene models to improve reconstruction.

III. METHOD

A. Basic scene representation

Scene representation: The 3D Gaussian distribution set is adopted. Each Gaussian point contains position $\mu \in R^3$, opacity $\alpha \in [0,1]$ and covariance matrix Σ . The covariance matrix Σ (Distinguish from the summation symbol in Eq.(2)) can be decomposed into through scaling S and rotation quaternions q :

$$\Sigma = RSS^T R^T \quad (1)$$

where R is the rotation matrix, represented by the rotation quaternion q . To reduce the amount of calculation, we abandoned the spherical harmonic function SH originally represented by the color C and synthesized the color C by mixing N Gaussian distributions:

$$C = \sum_{i \in N} c_i \cdot \alpha_i \cdot T_i \quad (2)$$

where $T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$ represents the probability that the light is not blocked before reaching the i -th Gaussian. As the number of Gauss that the light passes through increases, the occlusion effect is exhibited. Furthermore, during the rasterization process, the 3D Gaussian distribution (μ_w, Σ_w) in the world coordinate system also needs to be mapped onto the 2D image plane:

$$\mu_c = T_{cw} \mu_w \quad (3)$$

$$\Sigma_c = J W \Sigma_w W^T J^T \quad (4)$$

The transformation yields a 2D Gaussian distribution (μ_c, Σ_c) , where J is the Jacobian matrix of the linear approximation of the projection transformation, and W is the view transformation, mainly the rotation transformation here. This formula makes the 3D Gaussian distribution differentiable and realizes the differentiable rendering of 3D Gaussian.

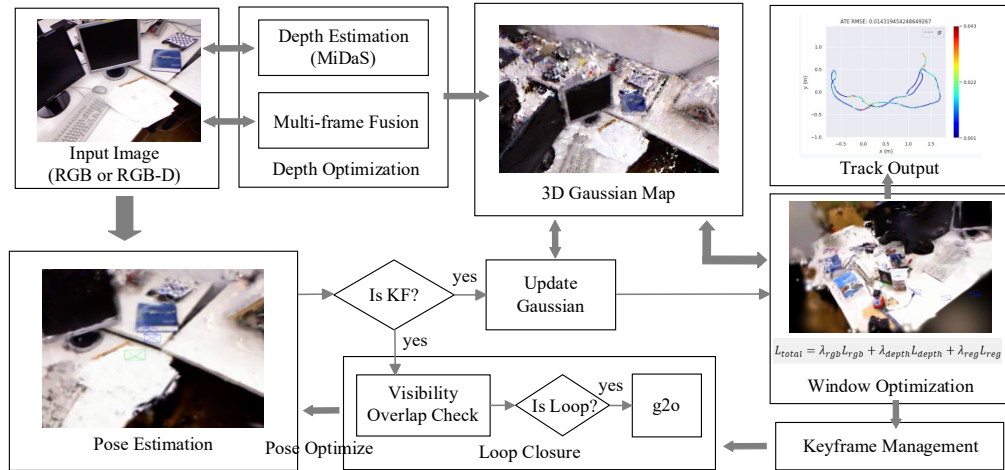


Figure 1. SLAM System Overview.

A front-end and back-end separated architecture is adopted: the front-end thread processes RGB or RGB-D input streams in real time, performing frame-to-frame tracking, keyframe selection, depth optimization, and loop closure detection; the back-end thread implements local mapping through a sliding window and conducts global consistency optimization based on the pose graph. Scene representation uses a collection of 3D Gaussian distributions, with the spatial distribution of Gaussian points supervised by differentiable rasterization rendering.

B. SLAM system

The overall system framework is illustrated in Fig. 1. In the following, we focus on the loop closure detection and depth optimization modules.

Loop Closure Detection: The detection part includes two stages: the initial screening of visibility similarity and the verification of geometric distance.

Initial screening of visibility similarity: For the current frame F_t , retrieve the candidate frame set C from the historical keyframe library H to satisfy the time interval constraint $|t-k| > \Delta_{min}$. Calculate the visibility mask overlap rate for each pair:

$$S_{vis}(c) = \frac{|V_t \cap V_c|}{\min(|V_t|, |V_c|)} \quad (5)$$

The candidate frames with $S_{vis} > \tau_1$ retained enter the geometric distance verification stage.

Geometric distance verification: For the candidate frames F_c that pass the initial screening, calculate the relative pose $T_{t \rightarrow c} = T_t W_c^{-1}$, and verify its translation component:

$$\|t_{t \rightarrow c}\|_2 < \tau_2 \quad (6)$$

Meanwhile, check the depth consistency of the common view area and eliminate abnormal matches.

Loop Frame Management: The selected candidate frames are ranked in descending order based on their overlap scores, and the top N_1 candidates are retained as the final loop closure candidates. This approach helps avoid mismatches caused by repetitive textures or environmental changes. Additionally, occlusion awareness is leveraged to significantly improve detection accuracy.

Pose graph optimization: After detecting the loop closure constraints (F_t, F_c) , construct a factor graph containing all keyframe nodes and loop closure edges, and optimize the objective function:

$$L_{pose} = \sum_{(i,j) \in E} \rho \left(\| \text{Log}(T_{ij}^{-1}(T_i^{-1}T_j)) \|_{\Sigma}^2 \right) \quad (7)$$

where ρ is the Huber robust kernel function, and E contains the odometer edge and the loop closure edge. LM optimization is achieved through the g2o library, and the world coordinates of Gaussian points are synchronously adjusted to maintain scene consistency.

Depth Optimization: When raw depth information is available, a multi-frame fusion approach is employed using the current frame and its neighboring frames to improve depth quality and reduce invalid depth values. Prior to fusion, each input depth map undergoes preprocessing with median filtering to suppress sensor noise. For the current frame I_c , a set of temporally adjacent frames $\{I_{c-i}, \dots, I_{c-1}, I_{c+1}, \dots, I_{c+i}\}$ (with a time interval of Δt) is selected as fusion candidates.

Multi-Frame Backprojection and Fusion: All neighboring frame depth maps are first backprojected into dense point clouds using the camera intrinsics. These point clouds are then transformed into the world coordinate system using their

respective pose extrinsics T_i . Subsequently, the points are projected back into the central frame's pixel coordinate system using the inverse pose T_c^{-1} , yielding corresponding pixel positions and depth values. This process is performed for each neighboring frame as follows:

$$P_{cam} = D(u, v) \cdot K^{-1} [u, v, 1], P_{world} = T_i \cdot P_{cam}^{hom} \quad (8)$$

$$P_c = K \cdot T_c^{-1} \cdot P_{world} \quad (9)$$

where K is the camera intrinsic matrix. To avoid fusion errors caused by incorrect projections or occlusions, a bidirectional consistency check is applied as a filtering step:

$$|\hat{D}_{i \rightarrow c} - D_i| < \epsilon \cdot D_i \quad (10)$$

Only when the projection from a neighboring frame to the central frame and then back to the original neighboring frame results in a reprojection error within a threshold ϵ , the pixel is considered reliable and included in the final depth fusion process.

Weighted Depth Fusion: For each projected pixel, a Gaussian-based depth difference weighting function is used to perform weighted fusion. This approach further suppresses the influence of inconsistent depth observations from neighboring frames. The weight is determined based on the depth difference between the central frame and the projected depth value:

$$w = \exp \left(-\frac{|D - D_c|}{\sigma \cdot D_c} \right) \quad (11)$$

where σ is the allowed error parameter. The final fused depth is computed as the weighted accumulation of all valid depths based on their respective weights:

$$D_{fused}(u, v) = \frac{\sum_i w_i \cdot D_i}{\sum_i w_i} \quad (12)$$

Finally, the grayscale image of the central frame is used as a guidance image to apply guided filtering on the fused depth map, further smoothing it and enhancing boundary accuracy.

In monocular settings where no ground-truth depth is available, we adopt the depth initialization strategy from MonoGS, which generates a pseudo-depth map based on the predicted or rendered depth and opacity from the 3D Gaussian Splatting model. However, this method is highly dependent on the accuracy and timeliness of the rendered scene. When the camera undergoes rapid viewpoint changes within a short time span, the predicted depth may become severely misaligned with the current image, resulting in significant rendering artifacts. Such distortions cannot be self-corrected within a single rendering cycle, compromising both visual quality and system stability.

To address this issue, we incorporate a CNN-based monocular depth prediction model—MiDaS [27]—into our system. Specifically, for frames where large viewpoint changes lead to mismatches between the rendered depth and the current image, we replace the distorted depth with the MiDaS-predicted depth map, effectively restoring rendering quality and improving system robustness.

IV. EXPERIMENT

A. Experimental Setup

Our method is implemented using the PyTorch framework with Python, and all experiments are conducted on a server equipped with a 16-core Intel(R) Xeon(R) Platinum 8481C CPU and a 24 GB NVIDIA RTX 4090 GPU. To evaluate the effectiveness of our approach in improving SLAM performance, we conduct extensive experiments focusing on both tracking accuracy and rendering quality.

The tracking evaluation is performed on static scenes from the TUM-RGBD dataset [30] and the Replica dataset [31], targeting pose accuracy in standard indoor environments. For rendering evaluation, we use both the TUM-RGBD dataset and an offline Realsense dataset. The latter is collected using a Realsense D455 sensor and includes raw RGB images, depth maps, and ground-truth camera poses, enabling a more intuitive assessment of rendering quality improvements. In addition, we demonstrate the enhanced robustness brought by our depth prediction strategy under monocular settings, particularly in cases where the camera undergoes rapid and large-angle viewpoint changes—scenarios where the original depth-from-rendering approach tends to fail.

B. Experimental evaluation

We evaluate camera tracking accuracy using the Root Mean Square Error (RMSE) of the Absolute Trajectory Error (ATE) computed on selected keyframes. For rendering quality, we adopt three standard photometric image quality metrics: Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index Measure (SSIM), and Learned Perceptual Image Patch Similarity (LPIPS), which collectively assess photometric fidelity and perceptual similarity of the rendered images. Where applicable, we compare our method against several state-of-the-art (SOTA) baselines to demonstrate its effectiveness and superiority in both tracking and rendering performance.

TABLE I. THIS METHOD WAS BENCHMARKED AGAINST OTHER METHODS ON THE TUM DATASET (RGBD), AND THE FIRST AND SECOND BEST RESULTS IN EACH SCENARIO WERE MARKED IN GREEN AND YELLOW RESPECTIVELY.

Class	Method	fr1/desk	fr2/xyz	fr3/office	Arg.
NeRF	iMAP	4.90	2.00	5.80	4.23
	NICE-SLAM	4.26	6.19	6.87	5.77
	Co-SLAM	2.40	1.70	2.40	2.17
	Point-SLAM	4.34	1.31	3.48	3.04
3DGS	SplaTAM[5]	3.35	1.24	5.16	3.25
	MonoGS	1.52	1.58	1.65	1.58
	GS-SLAM[32]	3.30	1.32	6.60	3.70
	Ours	1.43	1.20	1.39	1.34

TABLE II. THIS METHOD WAS BENCHMARKED AGAINST OTHER METHODS ON THE REPLICA DATASET, AND THE FIRST AND SECOND BEST RESULTS IN EACH SCENARIO WERE MARKED IN GREEN AND YELLOW RESPECTIVELY.

Class	Method	R0	R1	R2	O0	O1	Arg.
NeRF	iMAP	3.12	2.54	2.31	1.69	1.03	2.14
	NICE-SLAM	0.97	1.31	1.07	0.88	1.00	1.05
	Point-SLAM	0.61	0.41	0.37	0.38	0.48	0.45
3DGS	SplaTAM	0.36	0.31	0.40	0.29	0.47	0.37
	MonoGS	0.47	0.43	0.31	0.70	0.57	0.50
	GS-SLAM	0.48	0.53	0.33	0.52	0.41	0.45
	Ours	0.37	0.31	0.22	0.35	0.38	0.33

Tracking: As shown in Tables 1 and 2, our method consistently outperforms NeRF-SLAM across all evaluated scenes in both the TUM-RGBD and Replica datasets. On the TUM dataset, our approach achieves significantly better performance than all other 3DGS-SLAM methods (except MonoGS) in scenes such as fr1/desk and fr3/office, and slightly surpasses the best-performing SplaTAM in fr2/xyz. On the Replica dataset, our method performs slightly below SplaTAM in R0 and O0, but achieves the best performance in all other scenes, with results in R1 being nearly on par with SplaTAM. Overall, our system consistently maintains the lowest trajectory error across diverse datasets and scenes. The clear improvements over the next-best methods validate the effectiveness of our visibility-overlap-based loop closure detection and pose graph optimization using g2o, which play a pivotal role in enhancing the trajectory accuracy of Gaussian Splatting SLAM.

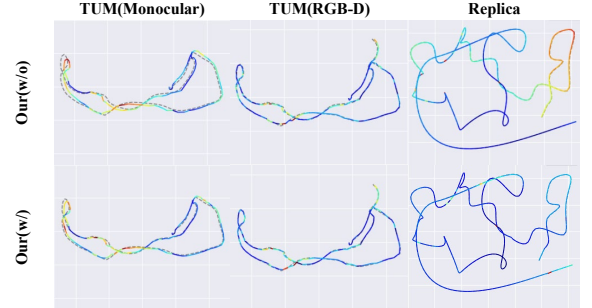


Figure 2. the system's trajectory accuracy on different datasets with and without loop closure detection.

In the figure, trajectories shown in bluer colors indicate higher accuracy, while redder colors indicate larger deviations. The dashed line represents the ground truth trajectory.

To further evaluate the effectiveness of the loop closure detection and pose graph optimization modules, we conduct an ablation study illustrated in Fig. 2, comparing camera trajectories with and without loop closure support. We specifically include experiments on the monocular-only TUM dataset to highlight the impact of loop closure under weak depth constraints. The results show that, in monocular settings where trajectory drift is more pronounced, the introduction of loop closure significantly improves trajectory alignment and loop consistency. Even in RGB-D settings where the baseline trajectory is already relatively accurate, the loop closure module still provides noticeable refinements and enhances global consistency.

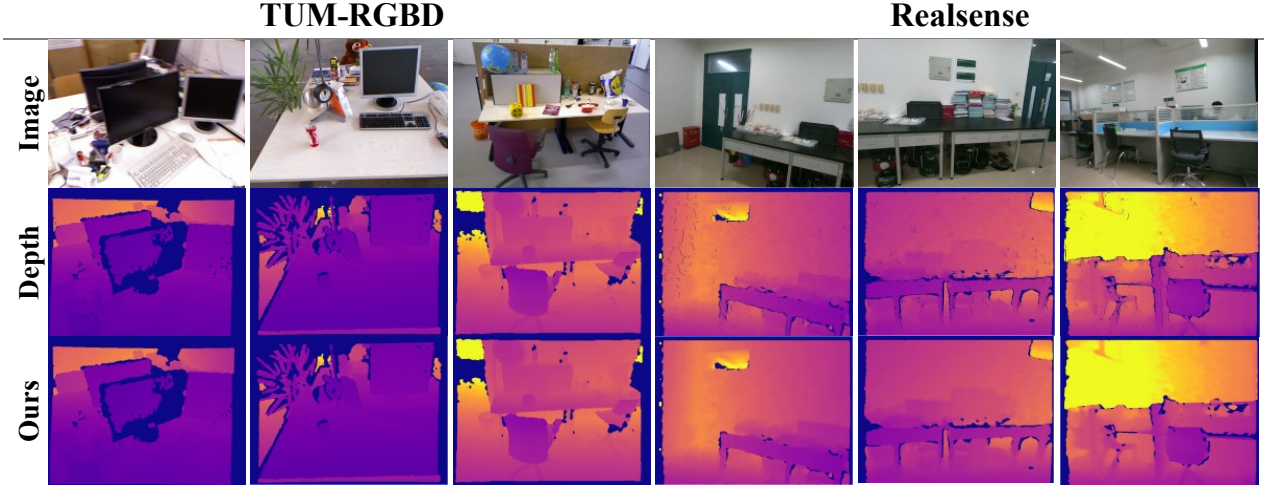


Figure 3. Comparison Before and After Multi-Frame Fusion.

We compare the effects of depth optimization before and after multi-frame fusion on three scenes from the TUM RGB-D dataset as well as on real-world scenes collected using the offline Realsense dataset.

TABLE III. RENDERING PERFORMANCE COMPARISON OF RGB-D SLAM METHODS ON THE TUM RGB-D DATASET: WE REPRODUCED THE RENDERING RESULTS OF VARIOUS METHODS ON THE TUM RGB-D DATASET FOR COMPARISON. THE FIRST AND SECOND BEST RESULTS IN EACH SCENARIO WERE MARKED IN RED AND GREEN RESPECTIVELY.

Method	Metric	fr1/desk	fr2/xyz	fr3/offi ce	Arg.
Point-SLAM	PSNR[dB]	13.74	17.68	18.26	16.56
	SSIM ↑	0.621	0.720	0.753	0.698
	LPIPS ↓	0.551	0.585	0.449	0.528
SplaTAM	PSNR[dB]	21.13	24.95	20.97	22.35
	SSIM ↑	0.837	0.948	0.864	0.883
	LPIPS ↓	0.259	0.103	0.223	0.195
MonoGS	PSNR[dB]	23.49	24.77	24.38	24.21
	SSIM ↑	0.781	0.795	0.825	0.800
	LPIPS ↓	0.249	0.217	0.219	0.228
Ours	PSNR[dB]	23.85	25.35	25.04	24.75
	SSIM ↑	0.786	0.816	0.837	0.813
	LPIPS ↓	0.252	0.198	0.214	0.221

Rendering: As illustrated in Fig. 3, we compare depth maps before and after multi-frame fusion on both the TUM-RGBD and offline Realsense datasets. The optimized depth maps exhibit significant improvements: invalid depth regions are greatly reduced, and object boundaries become notably clearer. Overall, the fused depth maps present higher density, better completeness, and improved geometric accuracy compared to their original counterparts. Table 3 provides quantitative evidence of our rendering quality improvements. Compared with NeRF-SLAM, 3DGS-SLAM demonstrates a clear advantage across all photometric metrics—PSNR, SSIM, and LPIPS. Our method also achieves comprehensive improvements over MonoGS, and surpasses SplaTAM in the fr1/desk and fr3/office scenarios. These performance gains can be attributed to the tight integration of loop closure detection and depth optimization. The proposed multi-frame depth fusion not only directly enhances rendering quality but also improves the accuracy of loop closure by providing more reliable depth information. In turn, accurate loop closures refine the global scene structure, leading to more consistent and higher-quality

renderings. This mutually reinforcing design forms a positive feedback loop that significantly boosts overall system performance.

As shown in Figure 4, under the monocular mode with only RGB input, we compare the impact of using MiDaS depth prediction on the rendering results during short-term, large-angle viewpoint changes. It can be clearly observed that relying solely on depth information generated from previously rendered scenes leads to severe rendering distortions under such drastic viewpoint variations. In contrast, replacing it with MiDaS-predicted depth maps results in more stable renderings, with significantly improved structural coherence and photometric consistency. This effectively alleviates the robustness issues of the original method in scenarios involving rapid viewpoint shifts. This experiment further validates the crucial role of the proposed depth prediction replacement strategy in enhancing system stability under dynamic viewpoint conditions.



Figure 4. Comparison of Scene Rendering in Monocular Mode With and Without MiDaS Depth Prediction.

The left image shows the original rendering result without MiDaS depth prediction, while the right image shows the result after incorporating MiDaS-predicted depth.

V. CONCLUSION

This paper presents a 3DGS-SLAM system that integrates visibility-overlap-based loop closure detection with multi-frame depth optimization. By fusing depth information from multiple frames, the system obtains more accurate geometric estimates. Loop closure candidates are selected using visibility mask overlap between Gaussian splatting, and further validated via relative pose constraints to reduce false positives. Upon successful loop detection, global pose graph optimization is

performed using g2o, followed by back-propagation of corrections to the 3D Gaussian point positions, ensuring global map consistency. To address depth distortion in monocular settings, we incorporate MiDaS for depth prediction, which significantly improves robustness under large viewpoint changes.

Extensive experiments on standard indoor RGB-D datasets and real-world Realsense recordings demonstrate that our system achieves improved tracking accuracy and rendering quality, even in challenging real-world conditions. Nonetheless, the current system has several limitations. The neighboring frame selection strategy remains relatively simple, and the optimized depth maps have yet to compensate for missing depth details caused by sensor occlusions. In future work, we plan to explore adaptive neighbor selection, completion of sparse or missing depth regions, and robust reconstruction under dynamic scenes to further enhance system performance and applicability.

REFERENCES

- [1] B. Kerbl, G. Kopanas, T. Leimkuhler, and G. Drettakis, “3D Gaussian splatting for real-time radiance field rendering,” *ACM Trans. Graph.*, vol. 42, no. 4, pp. 1–14, 2023.
- [2] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “NeRF: Representing scenes as neural radiance fields for view synthesis,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020, pp. 405–421.
- [3] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An open-source SLAM system for monocular, stereo, and RGB-D cameras,” *IEEE Trans. Robot.*, vol. 33, no. 5, pp. 1255–1262, Oct. 2017.
- [4] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A versatile and accurate monocular SLAM system,” *IEEE Trans. Robot.*, vol. 31, no. 5, pp. 1147–1163, Oct. 2015.
- [5] N. Keetha, J. Karhade, K. M. Jatavallabhula, G. Yang, S. Scherer, D. Ramanan, and J. Luiten, “SplaTAM: Splat track & map 3D Gaussians for dense RGB-D SLAM,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 21357–21366.
- [6] R. A. Newcombe, S. Izadi, O. Hilliges, D. Molyneaux, D. Kim, A. J. Davison, P. Kohli, J. Shotton, S. Hodges, and A. Fitzgibbon, “KinectFusion: Real-time dense surface mapping and tracking,” in *Proc. IEEE Int. Symp. Mixed Augmented Reality (ISMAR)*, 2011, pp. 127–136.
- [7] T. Whelan, S. Leutenegger, R. F. Salas-Moreno, B. Glocker, and A. J. Davison, “ElasticFusion: Dense SLAM without a pose graph,” in *Proc. Robotics: Science and Systems (RSS)*, 2015, Art. no. XI.001.
- [8] Z. Zhu, S. Peng, V. Larsson, W. Xu, H. Bao, Z. Cui, M. R. Oswald, and M. Pollefeys, “Nice-SLAM: Neural implicit scalable encoding for SLAM,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 12704–12713.
- [9] E. Sandstrom, Y. Li, L. Van Gool, and M. R. Oswald, “Point-SLAM: Dense neural point cloud-based SLAM,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023, pp. 2689–2699.
- [10] A. Dai, M. Nießner, M. Zollhöfer, S. Izadi, and C. Theobalt, “BundleFusion: Real-time globally consistent 3D reconstruction,” *ACM Trans. Graph.*, vol. 36, no. 4, pp. 1–18, Jul. 2017.
- [11] E. Sucar, S. Liu, J. Ortiz, and A. J. Davison, “iMAP: Implicit mapping and positioning in real-time,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021, pp. 6229–6238.
- [12] H. Wang, J. Wang, and L. Agapito, “Co-SLAM: Joint coordinate and sparse parametric encodings for neural real-time SLAM,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 13293–13302.
- [13] H. Li, X. Gu, W. Yuan, L. Yang, Z. Dong, and P. Tan, “Dense RGB SLAM with neural implicit map,” in *Int. Conf. Learn. Represent. (ICLR)*, 2023. [Online]. Available: arXiv:2301.08930
- [14] H. Matsuki, R. Murai, P. H. J. Kelly, and A. J. Davison, “Gaussian Splatting SLAM,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 18039–18048.
- [15] D. Galvez-López and J. D. Tardós, “Bags of binary words for fast place recognition in image sequences,” *IEEE Trans. Robot.*, vol. 28, no. 5, pp. 1188–1197, Oct. 2012.
- [16] R. Kümmerle, G. Grisetti, H. Strasdat, K. Konolige, and W. Burgard, “g2o: A general framework for graph optimization,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2011, pp. 3607–3613.
- [17] M. Cummins and P. Newman, “FAB-MAP: Probabilistic localization and mapping in the space of appearance,” *Int. J. Robot. Res.*, vol. 27, no. 6, pp. 647–665, 2008.
- [18] R. Arandjelović, P. Gronat, A. Torii, T. Pajdla, and J. Sivic, “NetVLAD: CNN architecture for weakly supervised place recognition,” in *Proc. IEEE Conf. Pattern Recognit. (CVPR)*, 2016, pp. 5297–5307.
- [19] N. Sünderhauf, S. Shirazi, A. Jacobson, F. Dayoub, E. Pepperell, B. Upcroft, and M. Milford, “Place recognition with ConvNet landmarks: Viewpoint-robust, condition-robust, training-free,” in *Proc. Robotics: Science and Systems (RSS)*, 2015, Art. no. XI.022.
- [20] L. Porzi, S. Rota Bulò, L. Leal-Taixé, and P. Kotschieder, “Learning multi-object tracking and segmentation from scratch,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 6846–6855.
- [21] A. Rosinol, M. Abate, Y. Chang, and L. Carlone, “Kimera: An open-source library for real-time metric-semantic localization and mapping,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2020, pp. 1689–1696.
- [22] J. Zhang and S. Singh, “LOAM: Lidar odometry and mapping in real-time,” in *Proc. Robotics: Science and Systems (RSS)*, 2014, Art. no. X.007.
- [23] J.-E. Deschaud, “IMLS-SLAM: Scan-to-model matching based on 3D data,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2018, pp. 2480–2487.
- [24] D. Eigen, C. Puhrsch, and R. Fergus, “Depth map prediction from a single image using a multi-scale deep network,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2014, pp. 2366–2374.
- [25] T. Zhou, M. Brown, N. Snaveley, and D. G. Lowe, “Unsupervised learning of depth and ego-motion from video,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 1851–1858.
- [26] C. Godard, O. Mac Aodha, and G. J. Brostow, “Unsupervised monocular depth estimation with left-right consistency,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 6602–6611.
- [27] R. Ranftl, A. Bochkovskiy, and V. Koltun, “Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 3, pp. 1623–1637, 2022.
- [28] Y. Yao, Z. Luo, S. Li, T. Fang, and L. Quan, “MVSNet: Depth inference for unstructured multi-view stereo,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 767–783.
- [29] B. Bescos, J. M. Fàcil, J. Civera, and J. Neira, “DynaSLAM: Tracking, mapping, and inpainting in dynamic scenes,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2018, pp. 4269–4276.
- [30] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, “A benchmark for the evaluation of RGB-D SLAM systems,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2012, pp. 573–580.
- [31] J. Straub, T. Whelan, L. Ma, Y. Chen, E. Wijmans, S. Green, J. J. Engel, R. Mur-Artal, C. Ren, S. Verma, A. Clarkson, M. Yan, B. Budge, Y. Yan, X. Pan, J. Yon, Y. Zou, K. Leon, N. Carter, J. Briales, T. Gillingham, E. Mueggler, L. Pesqueira, M. Savva, D. Batra, H. M. Strasdat, R. De Nardi, M. Goesele, S. Lovegrove, and R. Newcombe, “The Replica dataset: A digital replica of indoor spaces,” *arXiv preprint*, arXiv:1906.05797, 2019.
- [32] C. Yan, D. Qu, D. Xu, B. Zhao, Z. Wang, D. Wang, and X. Li, “GS-SLAM: Dense visual SLAM with 3D Gaussian Splatting,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 19595–19604.